

[USER] cat

DESCRIPTION : Affiche le contenu d'un ou plusieurs fichiers / concatène des fichiers  
 POURQUOI : Lecture rapide de fichiers, concaténation, redirection de contenu.  
 CONTEXTE : Lecture de configs, logs courts, création de fichiers par concaténation.  
 OPTIONS :

- n Numérote toutes les lignes
- b Numérote uniquement les lignes non vides
- A Affiche les caractères spéciaux (tabulations ^I, fin de ligne \$)
- s Réduit les lignes vides consécutives à une seule
- E Affiche \$ à la fin de chaque ligne
- T Affiche les tabulations comme ^I
- v Affiche les caractères non-affichables

EXEMPLES :

```
$ cat /etc/passwd
$ cat fichier1.txt fichier2.txt > fusion.txt
$ cat -A script.sh | grep '^M'
```

[USER] less

DESCRIPTION : Affiche un fichier page par page avec navigation bidirectionnelle  
 POURQUOI : Contrairement à more, less permet de remonter dans le fichier.  
 Idéal pour les fichiers volumineux – ne charge pas tout en mémoire.  
 CONTEXTE : Lecture de logs, man pages, fichiers de config volumineux.  
 OPTIONS :

- N Affiche les numéros de ligne
- i Recherche insensible à la casse
- S Tronque les longues lignes (pas de retour à la ligne)
- R Interprète les codes couleur ANSI
- F Quitte automatiquement si le fichier tient sur un écran
- X Ne vide pas l'écran à la sortie
- +motif Ouvre et cherche directement un motif
- +NUM Ouvre à la ligne NUM

NAVIGATION :

- Espace / f Page suivante
- b Page précédente
- /motif Chercher vers l'avant
- ?motif Chercher vers l'arrière
- n / N Résultat suivant / précédent
- g / G Début / fin du fichier
- q Quitter

EXEMPLES :

```
$ less /var/log/syslog
$ less -N +/ERROR /var/log/nginx/error.log
```

[USER] more

DESCRIPTION : Affiche un fichier page par page (navigation vers l'avant uniquement)  
 POURQUOI : Version simplifiée de less – présent sur tous les systèmes Unix.  
 CONTEXTE : Systèmes minimaux, compatibilité universelle.  
 OPTIONS :

- d Affiche des instructions de navigation
- NUM Définit la taille de page à NUM lignes
- +motif Commence à la première occurrence du motif
- +NUM Commence à la ligne NUM

EXEMPLES :

```
$ more /etc/services
$ cat fichier.txt | more
```

[USER] head

DESCRIPTION : Affiche les premières lignes d'un fichier  
 POURQUOI : Voir le début d'un fichier (en-tête CSV, début de log, format de fichier).  
 CONTEXTE : Inspection rapide, scripts de traitement de données.  
 OPTIONS :

- n N Affiche les N premières lignes (défaut: 10)

```

-c N      Affiche les N premiers octets
-n N      Affiche tout SAUF les N dernières lignes
-q        Pas d'en-tête de nom de fichier
-v        Toujours afficher l'en-tête de nom de fichier

```

EXEMPLES :

```

$ head /var/log/syslog
$ head -n 5 fichier.csv
$ head -c 100 binaire.bin | xxd

```

```
[USER] tail
```

DESCRIPTION : Affiche les dernières lignes d'un fichier – avec suivi en temps réel

POURQUOI : Surveiller des logs en direct, voir les derniers événements système.  
-f est l'une des options les plus utilisées en administration.

CONTEXTE : Monitoring de logs, débogage en temps réel, suivi d'applications.

OPTIONS :

```

-n N      Affiche les N dernières lignes (défaut: 10)
-c N      Affiche les N derniers octets
-f        Suit le fichier en temps réel (follow)
-F        Comme -f mais recrée le fichier si rotaté
--pid=PID Arrête le suivi quand le processus PID se termine
-s N      Intervalle de vérification en secondes (avec -f)
-n +N     Affiche depuis la ligne N jusqu'à la fin
-q        Pas d'en-tête de nom de fichier

```

EXEMPLES :

```

$ tail -f /var/log/nginx/access.log
$ tail -n 50 /var/log/messages
$ tail -F /var/log/app.log --pid=$(pgrep monapp)

```

```
[USER] grep
```

DESCRIPTION : Recherche de motifs (expressions régulières) dans des fichiers ou flux

POURQUOI : Filtrer des informations dans des fichiers ou la sortie d'autres commandes.  
Outil indispensable – utilisé des centaines de fois par jour.

CONTEXTE : Recherche dans logs, filtrage de sorties, audit de configs, scripting.

OPTIONS :

```

-i        Insensible à la casse
-v        Inverse : affiche les lignes qui NE correspondent PAS
-r / -R   Recherche récursive dans les répertoires
-l        Affiche uniquement les noms de fichiers correspondants
-L        Affiche les fichiers sans correspondance
-n        Affiche le numéro de ligne
-c        Affiche le nombre de correspondances
-o        Affiche uniquement la partie correspondante
-A N      Affiche N lignes après la correspondance
-B N      Affiche N lignes avant la correspondance
-C N      Affiche N lignes avant ET après
-E        Utilise les expressions régulières étendues (ERE)
-F        Traite le motif comme une chaîne fixe (pas de regex)
-P        Utilise les expressions régulières Perl (PCRE)
-w        Correspond aux mots entiers uniquement
-x        Correspond aux lignes entières uniquement
-m N      Arrête après N correspondances
--color=auto Colorise les correspondances
-H        Toujours afficher le nom du fichier
-h        Ne jamais afficher le nom du fichier

```

EXEMPLES :

```

$ grep -rn "password" /etc/
$ grep -v "^#" /etc/nginx/nginx.conf
$ grep -E "ERROR|WARN" /var/log/app.log
$ grep -A 3 -B 1 "Exception" /var/log/java.log
$ ps aux | grep -v grep | grep nginx

```

```
[USER] egrep
```

DESCRIPTION : Comme grep mais avec les expressions régulières étendues par défaut

POURQUOI : Équivalent à grep -E – syntaxe plus lisible pour les regex complexes.

CONTEXTE : Filtrage avancé avec des regex contenant |, +, ?, {}, ().

EXEMPLES :

```

$ egrep "error|warning|critical" /var/log/syslog
$ egrep "[0-9]{1,3}\.[0-9]{1,3}" access.log

```

```
[USER] fgrep
```

DESCRIPTION : Comme grep mais traite le motif comme une chaîne fixe (pas de regex)  
POURQUOI : Plus rapide quand on cherche une chaîne exacte sans caractères spéciaux.  
Évite les problèmes d'échappement de caractères regex.  
CONTEXTE : Recherche de chaînes exactes, recherche d'IP, d'URLs, de chemins.  
EXEMPLES :  
\$ fgrep "192.168.1.1" /var/log/auth.log  
\$ fgrep ".php" access.log

```
[USER] sed
```

DESCRIPTION : Éditeur de flux (Stream Editor) – transforme du texte ligne par ligne  
POURQUOI : Remplacement, suppression, insertion de texte dans des fichiers ou flux.  
L'un des outils les plus puissants du shell.  
CONTEXTE : Modification de configs, nettoyage de données, scripts de transformation.  
OPTIONS :  
-n Supprime l'affichage automatique (nécessite p pour afficher)  
-i Modification en place du fichier  
-i.bak Modification en place avec sauvegarde  
-e 'SCRIPT' Spécifie plusieurs expressions  
-f FICHIER Lit les commandes depuis un fichier  
-r / -E Utilise les expressions régulières étendues  
COMMANDES sed:  
s/motif/remplacement/ Substitution (première occurrence par ligne)  
s/motif/remplacement/g Substitution globale (toutes occurrences)  
s/motif/remplacement/i Substitution insensible à la casse  
d Supprime la ligne  
p Affiche la ligne  
q Quitte après la première correspondance  
a\texte Ajoute une ligne après  
i\texte Insère une ligne avant  
NUM Applique à la ligne NUM  
/motif/ Applique aux lignes correspondant au motif  
NUM1,NUM2 Plage de lignes  
EXEMPLES :  
\$ sed 's/ancien/nouveau/g' fichier.txt  
\$ sed -i.bak 's/localhost/0.0.0.0/' /etc/redis.conf  
\$ sed -n '10,20p' fichier.txt  
\$ sed '/^#/d' /etc/nginx/nginx.conf  
\$ sed -i 's/^SELINUX=.\*SELINUX=disabled/' /etc/selinux/config

```
[USER] awk
```

DESCRIPTION : Langage de traitement de texte structuré orienté colonnes et champs  
POURQUOI : Traiter des fichiers tabulaires (CSV, logs formatés), extraire des colonnes,  
effectuer des calculs, générer des rapports. Bien plus puissant que cut.  
CONTEXTE : Traitement de logs, analyse de données, génération de rapports, scripting.  
OPTIONS :  
-F SEP Définit le séparateur de champ (défaut: espace)  
-v VAR=VAL Définit une variable avant exécution  
-f FICHIER Lit le programme awk depuis un fichier  
VARIABLES INTERNES :  
\$0 Ligne entière  
\$1, \$2, ... Champs 1, 2, ...  
NF Nombre de champs dans la ligne courante  
NR Numéro de ligne courant  
FS Séparateur de champ (Field Separator)  
OFS Séparateur de champ en sortie  
RS Séparateur de ligne  
FILENAME Nom du fichier en cours  
EXEMPLES :  
\$ awk -F: '{print \$1}' /etc/passwd  
\$ awk '{print \$1, \$4}' access.log  
\$ awk -F: '\$3 >= 1000 {print \$1}' /etc/passwd  
\$ awk 'NR==5,NR==10' fichier.txt  
\$ awk '{sum += \$1} END {print "Total:", sum}' nombres.txt  
\$ ps aux | awk '{print \$2, \$11}' | grep nginx

```
[USER] gawk
```

DESCRIPTION : GNU awk – implémentation GNU de awk avec fonctionnalités étendues  
POURQUOI : Fonctionnalités supplémentaires par rapport à awk standard : tableaux multidimensionnels, fonctions intégrées étendues, réseau.  
CONTEXTE : Scripts awk avancés, traitement de données complexes.  
EXEMPLES :  
\$ gawk 'BEGIN{PROCINFO["sorted\_in"]="@val\_num\_desc"} {a[\$1]=\$2} END{for(k in a) print k,a[k]}' f.txt

[USER] cut

DESCRIPTION : Extrait des colonnes ou champs d'un fichier texte  
POURQUOI : Extraction simple et rapide de colonnes depuis des fichiers CSV ou des sorties de commandes.  
CONTEXTE : Extraction de colonnes de CSV, traitement de sorties de commandes.  
OPTIONS :  
-d SEP Définit le délimiteur de champ  
-f N Extrait le champ N  
-f N-M Extrait les champs N à M  
-f N,M Extrait les champs N et M  
-c N Extrait le caractère N  
-c N-M Extrait les caractères N à M  
--complement Affiche tout SAUF la sélection  
EXEMPLES :  
\$ cut -d: -f1 /etc/passwd  
\$ cut -d, -f2,4 donnees.csv  
\$ cut -c1-80 fichier\_large.txt

[USER] sort

DESCRIPTION : Trie les lignes d'un fichier texte  
POURQUOI : Ordonner des données, préparer pour uniq, créer des listes triées.  
CONTEXTE : Traitement de données, génération de rapports, dédoublonnage.  
OPTIONS :  
-n Tri numérique (pas lexicographique)  
-r Tri inverse  
-u Supprime les doublons (unique)  
-k N Trie selon le champ N  
-k N,M Trie selon les champs N à M  
-t SEP Définit le séparateur de champ  
-f Insensible à la casse  
-h Tri de tailles humaines (1K, 2M, 3G)  
-V Tri de versions (1.2 < 1.10)  
-R Ordre aléatoire  
-m Fusionne des fichiers déjà triés  
-o FICHIER Écrit dans un fichier  
--parallel=N Utilise N threads  
EXEMPLES :  
\$ sort -n nombres.txt  
\$ sort -t: -k3 -n /etc/passwd  
\$ sort -rh /tmp/tailles.txt  
\$ sort -u liste.txt

[USER] uniq

DESCRIPTION : Supprime ou compte les lignes consécutives identiques  
POURQUOI : Dédoublonnage de données, comptage de fréquences. À combiner avec sort.  
CONTEXTE : Analyse de fréquence dans les logs, dédoublonnage, statistiques.  
OPTIONS :  
-c Préfixe chaque ligne par son nombre d'occurrences  
-d Affiche seulement les lignes dupliquées  
-u Affiche seulement les lignes uniques  
-i Insensible à la casse  
-f N Ignore les N premiers champs  
-s N Ignore les N premiers caractères  
-w N Compare uniquement les N premiers caractères  
EXEMPLES :  
\$ sort fichier.txt | uniq  
\$ sort access.log | uniq -c | sort -rn | head -20  
\$ sort -u liste.txt

[USER] wc

DESCRIPTION : Compte les lignes, mots, caractères et octets d'un fichier  
 POURQUOI : Statistiques rapides sur un fichier ou la sortie d'une commande.  
 CONTEXTE : Vérification de taille, comptage de résultats, scripts.  
 OPTIONS :

- l Compte les lignes uniquement
- w Compte les mots uniquement
- c Compte les octets uniquement
- m Compte les caractères (multi-octets)
- L Longueur de la ligne la plus longue

EXEMPLES :

```
$ wc -l /etc/passwd
$ wc -w rapport.txt
$ grep ERROR /var/log/app.log | wc -l
```

[USER] tr

DESCRIPTION : Traduit, supprime ou compresse des caractères dans un flux  
 POURQUOI : Conversion de casse, suppression de caractères, remplacement simple.  
 Travaile sur des caractères individuels (pas des chaînes comme sed).  
 CONTEXTE : Nettoyage de données, conversion de casse, suppression de caractères spéciaux.  
 OPTIONS :

- d Supprime les caractères de l'ensemble 1
- s Comprime les répétitions en un seul caractère
- c Complément de l'ensemble 1

CLASSES :

- [:upper:] Lettres majuscules
- [:lower:] Lettres minuscules
- [:digit:] Chiffres
- [:space:] Espaces et tabulations
- [:punct:] Ponctuation

EXEMPLES :

```
$ echo "HELLO" | tr '[:upper:]' '[:lower:]'
$ tr -d '\r' < fichier_windows.txt > fichier_unix.txt
$ echo "hello world" | tr -s ' '
$ tr -cd '[:print:]' < binaire.bin
```

[USER] tee

DESCRIPTION : Lit depuis stdin et écrit vers stdout ET vers un ou plusieurs fichiers  
 POURQUOI : Sauvegarder la sortie d'une commande tout en continuant à la pipe-r.  
 Voir la sortie à l'écran ET la sauvegarder en même temps.  
 CONTEXTE : Logging de sorties, débogage de pipelines complexes.  
 OPTIONS :

- a Ajoute au fichier au lieu d'écraser
- i Ignore les signaux d'interruption

EXEMPLES :

```
$ make 2>&1 | tee build.log
$ curl -s https://example.com | tee output.html | grep "<title>"
$ commande | tee -a /var/log/monapp.log | grep ERROR
```

[USER] echo

DESCRIPTION : Affiche une ligne de texte ou des variables  
 POURQUOI : Affichage dans les scripts, création de contenu, débogage de variables.  
 CONTEXTE : Scripts shell omniprésent – affichage de messages, création de fichiers.  
 OPTIONS :

- n Ne pas ajouter de saut de ligne final
- e Interprète les séquences d'échappement (\n, \t, \a...)
- E Désactive l'interprétation des séquences (défaut)

SÉQUENCES (-e):

- \n Saut de ligne
- \t Tabulation
- \a Son d'alerte (bell)
- \b Retour arrière
- \c Supprime tout le reste de la sortie
- \\ Backslash

EXEMPLES :

```
$ echo "Bonjour $USER"
$ echo -e "Ligne1\nLigne2\nLigne3"
$ echo -n "Sans saut de ligne"
$ echo "contenu" > fichier.txt
```

```
[USER] printf
```

DESCRIPTION : Affiche du texte formaté selon un format de type C printf()  
POURQUOI : Plus portable et prévisible qu'echo. Formatage précis de nombres, largeurs de colonnes, padding.  
CONTEXTE : Scripts portables, formatage de tableaux, affichage de données alignées.  
OPTIONS :  
    %s Chaîne de caractères  
    %d Entier décimal  
    %f Nombre flottant  
    %x Hexadécimal  
    %o Octal  
    %05d Entier sur 5 chiffres avec zéros  
    %-20s Chaîne alignée à gauche sur 20 caractères  
EXEMPLES :  
\$ printf "%-20s %5d\n" "Total:" 42  
\$ printf "%x\n" 255  
\$ printf "%.2f\n" 3.14159  
\$ printf '%s\n' "Ligne1" "Ligne2" "Ligne3"

```
[USER] diff
```

DESCRIPTION : Compare deux fichiers ligne par ligne et affiche les différences  
POURQUOI : Voir ce qui a changé entre deux versions d'un fichier, comparer des configs.  
CONTEXTE : Revue de code, comparaison de configs, génération de patches.  
OPTIONS :  
    -u Format unifié (le plus lisible, utilisé par git)  
    -c Format contextuel  
    -y Format côte à côte  
    -i Insensible à la casse  
    -b Ignore les différences d'espaces  
    -B Ignore les lignes vides  
    -r Compare récursivement des répertoires  
    -q Indique seulement si les fichiers diffèrent  
    -N Traite les fichiers absents comme vides  
    --color Colorise la sortie  
EXEMPLES :  
\$ diff fichier\_original.conf fichier\_modifie.conf  
\$ diff -u config.old config.new > mon.patch  
\$ diff -r /etc/nginx/ /backup/nginx/  
\$ diff -y fichier1.txt fichier2.txt

```
[USER] diff3
```

DESCRIPTION : Compare trois fichiers simultanément (utile pour les merges)  
POURQUOI : Résoudre des conflits de fusion entre trois versions d'un fichier.  
CONTEXTE : Gestion de version, résolution de conflits, merge de configs.  
OPTIONS :  
    -m Génère un fichier fusionné  
    -A Montre tous les changements  
    -e Génère des instructions ed  
EXEMPLES :  
\$ diff3 mon\_fichier original version\_distante

```
[USER] sdiff
```

DESCRIPTION : Compare deux fichiers côte à côte et permet une fusion interactive  
POURQUOI : Interface plus visuelle que diff pour voir les différences.  
CONTEXTE : Fusion interactive de fichiers, comparaison visuelle.  
OPTIONS :  
    -w N Largeur de la sortie  
    -i Insensible à la casse  
    -s Supprime les lignes identiques  
    -o SORTIE Fusion interactive vers un fichier  
EXEMPLES :  
\$ sdiff fichier1.txt fichier2.txt  
\$ sdiff -o fusion.txt original.txt modifie.txt

```
[USER] comm
```

DESCRIPTION : Compare deux fichiers triés et affiche les lignes communes/différentes  
 POURQUOI : Trouver l'intersection ou la différence entre deux listes triées.  
 CONTEXTE : Comparaison de listes, audit de fichiers, différences entre inventaires.  
 OPTIONS :  
 -1 Supprime colonne 1 (lignes uniques au fichier 1)  
 -2 Supprime colonne 2 (lignes uniques au fichier 2)  
 -3 Supprime colonne 3 (lignes communes)  
 -i Insensible à la casse  
 EXEMPLES :  
 \$ comm -12 <(sort liste1.txt) <(sort liste2.txt)  
 \$ comm -23 <(sort liste1.txt) <(sort liste2.txt)

[USER] paste

DESCRIPTION : Fusionne des fichiers ligne par ligne (colle les colonnes côte à côte)  
 POURQUOI : Combiner plusieurs colonnes de différents fichiers en un seul tableau.  
 CONTEXTE : Assemblage de données tabulaires, création de CSV.  
 OPTIONS :  
 -d SEP Utilise SEP comme délimiteur (défaut: tabulation)  
 -s Fusionne lignes d'un même fichier en une seule ligne  
 EXEMPLES :  
 \$ paste noms.txt prenom.txt ages.txt  
 \$ paste -d, coll.txt col2.txt col3.txt > tableau.csv  
 \$ paste -s fichier.txt

[USER] join

DESCRIPTION : Joint deux fichiers triés sur un champ commun (comme un JOIN SQL)  
 POURQUOI : Combiner des données de deux fichiers ayant une clé commune.  
 CONTEXTE : Traitement de données relationnelles en shell.  
 OPTIONS :  
 -1 N Champ de jointure dans le fichier 1  
 -2 N Champ de jointure dans le fichier 2  
 -t SEP Délimiteur de champ  
 -a N Affiche aussi les lignes non jointes du fichier N  
 -v N Affiche uniquement les lignes non jointes du fichier N  
 -o FORMAT Spécifie les champs à afficher  
 -i Insensible à la casse  
 EXEMPLES :  
 \$ join -t, fichier1.csv fichier2.csv  
 \$ join -1 2 -2 1 employes.txt salaires.txt

[USER] column

DESCRIPTION : Formate la sortie en colonnes alignées  
 POURQUOI : Rendre lisible des données tabulaires dans le terminal.  
 CONTEXTE : Affichage de tableaux, formatage de sorties de commandes.  
 OPTIONS :  
 -t Crée un tableau avec colonnes alignées  
 -s SEP Délimiteur d'entrée  
 -o SEP Délimiteur de sortie  
 -n Ne fusionne pas les délimiteurs vides  
 -J Sortie en JSON  
 -x Remplit les colonnes horizontalement  
 EXEMPLES :  
 \$ mount | column -t  
 \$ cat /etc/passwd | cut -d: -f1,3,7 | column -t -s:  
 \$ df -h | column -t

[USER] fmt

DESCRIPTION : Formate le texte en paragraphes avec une largeur maximale de ligne  
 POURQUOI : Reformater du texte qui dépasse la largeur voulue (emails, docs).  
 CONTEXTE : Mise en forme de documents texte, reformatage de paragraphes.  
 OPTIONS :  
 -w N Largeur maximale (défaut: 75)  
 -s Ne joint pas les courtes lignes  
 -u Un espace entre les mots  
 EXEMPLES :  
 \$ fmt -w 80 document.txt  
 \$ cat README | fmt -w 72

```
[USER] fold
```

DESCRIPTION : Coupe les longues lignes à une largeur spécifiée  
POURQUOI : Adapter du texte à une largeur d'écran ou de terminal.  
CONTEXTE : Préparation de texte pour impression, terminaux étroits.  
OPTIONS :  
-w N Largeur de ligne (défaut: 80)  
-s Coupe aux espaces (pas en milieu de mot)  
-b Compte en octets plutôt qu'en colonnes  
EXEMPLES :  
\$ fold -w 72 -s long\_texte.txt  
\$ echo "ligne tres longue..." | fold -w 40

```
[USER] expand / unexpand
```

DESCRIPTION : expand : convertit les tabulations en espaces  
unexpand : convertit les espaces en tabulations  
POURQUOI : Normaliser l'indentation, corriger des problèmes de tabulations dans du code.  
CONTEXTE : Préparation de code, interopérabilité entre éditeurs.  
OPTIONS :  
-t N Taille de tabulation en espaces (défaut: 8)  
-i (unexpand) Convertit seulement l'indentation initiale  
EXEMPLES :  
\$ expand -t 4 code.py  
\$ unexpand -t 4 fichier.txt

```
[USER] nl
```

DESCRIPTION : Numérote les lignes d'un fichier  
POURQUOI : Ajouter des numéros de ligne à un fichier pour référencer des positions.  
CONTEXTE : Documentation, débogage, référencement de code.  
OPTIONS :  
-b a Numérote toutes les lignes (y compris vides)  
-b t Numérote uniquement les lignes non vides (défaut)  
-n rz Format avec zéros: 000001  
-n ln Format aligné à gauche  
-s SEP Séparateur après le numéro  
-v N Commence à N  
-w N Largeur du numéro  
EXEMPLES :  
\$ nl -b a fichier.txt  
\$ nl -n rz -w 4 script.sh

```
[USER] rev
```

DESCRIPTION : Inverse l'ordre des caractères de chaque ligne  
POURQUOI : Traiter des données dans l'ordre inversé, extraire les dernières colonnes.  
CONTEXTE : Scripts de transformation de données, extraction du dernier champ.  
EXEMPLES :  
\$ echo "bonjour" | rev  
\$ rev fichier.txt  
\$ echo "a:b:c:d" | rev | cut -d: -f1 | rev

```
[USER] tac
```

DESCRIPTION : Affiche un fichier en ordre inversé (cat à l'envers – dernière ligne d'abord)  
POURQUOI : Voir la fin d'un log en premier, inverser l'ordre de traitement.  
CONTEXTE : Analyse de logs (voir les dernières entrées en premier), scripts.  
OPTIONS :  
-s SEP Utilise SEP comme séparateur de ligne  
-r Interprète SEP comme une expression régulière  
EXEMPLES :  
\$ tac /var/log/messages | less  
\$ tac fichier.txt

```
[USER] od
```

DESCRIPTION : Affiche le contenu d'un fichier en octal, hexadécimal, ASCII ou autre



POURQUOI : Inspecter le contenu binaire d'un fichier, voir les caractères cachés, analyser des protocoles ou formats binaires.

CONTEXTE : Analyse forensics, débogage de fichiers binaires, analyse de protocoles.

OPTIONS :

- c Affiche les caractères avec échappements
- x Affiche en hexadécimal 2 octets
- An Pas d'adresses
- Ax Adresses en hexadécimal
- t x1 Hexadécimal octet par octet
- t d1 Décimal octet par octet
- v Affiche les données dupliquées
- N N Affiche seulement N octets
- j N Saute N octets au début

EXEMPLES :

```
$ od -c /bin/ls | head
$ od -t x1 fichier.bin
$ od -An -t x1 -N 16 /dev/urandom
```

[USER] hexdump

DESCRIPTION : Affiche le contenu d'un fichier en hexadécimal avec représentation ASCII

POURQUOI : Analyser un exécutable, trouver des chemins codés en dur, des mots de passe, des URLs dans un binaire.

CONTEXTE : Analyse binaire, débogage de protocoles, sécurité informatique.

OPTIONS :

- C Format canonique: hex + ASCII côte à côte (le plus utile)
- n N Lit seulement N octets
- s N Saute N octets
- e FORMAT Format personnalisé
- v Affiche les données dupliquées

EXEMPLES :

```
$ hexdump -C binaire.exe | head -20
$ hexdump -C -n 512 /dev/sda
$ hexdump -n 4 -e '1/4 "Magic: 0x%08x\n"' fichier
```

[USER] strings

DESCRIPTION : Extrait les chaînes de caractères lisibles d'un fichier binaire

POURQUOI : Analyser un exécutable, trouver des chemins codés en dur, des mots de passe, des URLs dans un binaire.

CONTEXTE : Sécurité, analyse de malware, reverse engineering, forensics.

OPTIONS :

- n N Longueur minimale des chaînes (défaut: 4)
- a Analyse tout le fichier (pas seulement les sections data)
- t x Affiche l'offset hexadécimal
- t d Affiche l'offset décimal
- e b Chaînes 16-bit big-endian
- e l Chaînes 16-bit little-endian

EXEMPLES :

```
$ strings /bin/bash | grep "version"
$ strings -n 8 binaire_suspect | grep -i "password"
$ strings firmware.bin | grep -E "http|ftp|ssh"
```

[USER] xargs

DESCRIPTION : Construit et exécute des commandes à partir de l'entrée standard

POURQUOI : Passer la sortie d'une commande comme arguments d'une autre. Résout la limite du shell sur le nombre d'arguments.

CONTEXTE : Pipelines complexes, traitement en masse, scripts d'administration.

OPTIONS :

- n N Passe N arguments à la fois
- I {} Remplace {} par l'argument dans la commande
- P N Exécute N processus en parallèle
- 0 Utilise \0 comme séparateur (avec find -print0)
- r N'exécute pas si stdin est vide
- t Affiche la commande avant exécution
- a FICHIER Lit depuis un fichier au lieu de stdin
- L N Passe N lignes à la fois
- max-args=N Alias de -n

EXEMPLES :

```
$ find . -name "*.log" -print0 | xargs -0 rm
$ cat urls.txt | xargs -P 4 -I {} wget {}
$ find . -name "*.py" | xargs grep -l "import os"
```

```
$ echo "a b c" | xargs -n1 echo
```

```
[USER] bc
```

DESCRIPTION : Calculatrice en précision arbitraire (Basic Calculator)  
POURQUOI : Effectuer des calculs mathématiques dans le shell, y compris flottants.  
bash ne gère pas les flottants nativement.  
CONTEXTE : Scripts de calcul, conversions, calculs de tailles.  
OPTIONS :  
-l Charge la bibliothèque mathématique standard (sin, cos...)  
-i Mode interactif  
-q Mode silencieux (pas de bannière)  
scale=N Définit la précision décimale  
EXEMPLES :  
\$ echo "scale=4; 22/7" | bc  
\$ echo "sqrt(2)" | bc -l  
\$ echo "obase=16; 255" | bc  
\$ echo "ibase=16; FF" | bc

```
[USER] dc
```

DESCRIPTION : Calculatrice en notation polonaise inverse (RPN – Reverse Polish Notation)  
POURQUOI : Calculs en notation postfixe – utilisé dans les scripts pour sa rapidité.  
CONTEXTE : Scripts avancés, calculs en notation RPN.  
EXEMPLES :  
\$ echo "5 3 + p" | dc  
\$ echo "2 8 ^ p" | dc

```
[USER] expr
```

DESCRIPTION : Évalue des expressions arithmétiques, de comparaison ou de chaînes  
POURQUOI : Calculs simples dans les scripts shell sans sous-shell.  
CONTEXTE : Scripts shell anciens, calculs d'index, comparaisons.  
EXEMPLES :  
\$ expr 5 + 3  
\$ expr length "bonjour"  
\$ expr substr "bonjour" 2 4  
\$ N=\$(expr \$N + 1)

```
[USER] factor
```

DESCRIPTION : Décompose un nombre en facteurs premiers  
POURQUOI : Utilitaire mathématique, utilisé en cryptographie et scripts.  
CONTEXTE : Mathématiques, scripts éducatifs, cryptographie.  
EXEMPLES :  
\$ factor 360  
\$ factor 2147483647

```
[USER] numfmt
```

DESCRIPTION : Convertit des nombres vers/depuis des formats lisibles (K, M, G...)  
POURQUOI : Formater des tailles ou des chiffres pour l'affichage humain ou automatique.  
CONTEXTE : Scripts de monitoring, affichage de métriques.  
OPTIONS :  
--to=si Vers unités SI (K=1000)  
--to=iec Vers unités IEC (K=1024)  
--from=si Depuis unités SI  
--from=iec Depuis unités IEC  
--field=N Convertit seulement le champ N  
--suffix=SUF Ajoute un suffixe  
EXEMPLES :  
\$ numfmt --to=iec 1073741824  
\$ df -B1 | numfmt --field=2,3,4 --from-unit=1 --to=iec

```
[USER] seq
```

DESCRIPTION : Génère une séquence de nombres  
POURQUOI : Créer des boucles numériques, générer des listes de numéros.  
CONTEXTE : Scripts de boucle, génération de données de test.

```

OPTIONS      :
  -w          Pad avec des zéros pour alignement
  -s SEP      Séparateur (défaut: newline)
  -f FORMAT   Format printf
EXEMPLES     :
  $ seq 1 10
  $ seq 0 2 20
  $ seq -w 1 100
  $ for i in $(seq 1 5); do echo "Itération $i"; done

```

```
[USER] shuf
```

```

DESCRIPTION  : Mélange aléatoirement les lignes d'un fichier ou génère des nombres aléatoires
POURQUOI    : Sélection aléatoire, tests avec données aléatoires, tirage au sort.
CONTEXTE    : Tests, sélection aléatoire, jeux, échantillonnage.
OPTIONS      :
  -n N        Retourne seulement N lignes
  -r          Avec remplacement (une ligne peut apparaître plusieurs fois)
  -i MIN-MAX  Génère des entiers aléatoires dans l'intervalle
  -o FICHIER  Écrit dans un fichier
  -e ARGS     Traite les arguments comme des lignes
EXEMPLES     :
  $ shuf liste.txt | head -5
  $ shuf -i 1-100 -n 10
  $ shuf -e "rouge" "vert" "bleu" | head -1

```

```
[USER] tsort
```

```

DESCRIPTION  : Tri topologique – ordonne les éléments selon leurs dépendances
POURQUOI    : Résoudre l'ordre d'exécution de tâches avec dépendances.
CONTEXTE    : Résolution de dépendances, ordonnancement de tâches.
EXEMPLES     :
  $ echo -e "A B\nB C\nA C" | tsort

```

```
[USER] look
```

```

DESCRIPTION  : Recherche des lignes dans un fichier trié commençant par une chaîne
POURQUOI    : Recherche rapide dans un dictionnaire ou fichier trié.
CONTEXTE    : Vérification orthographique, recherche dans des listes triées.
OPTIONS      :
  -d          Compare seulement les caractères alphanumériques
  -f          Insensible à la casse
  -t SEP      Caractère de terminaison
EXEMPLES     :
  $ look "inter" /usr/share/dict/words
  $ look "192.168" /etc/hosts

```

```
[USER] col / colcrt / colrm
```

```

DESCRIPTION  : col : filtre les séquences de contrôle de terminal
               colcrt : filtre pour affichage sur terminaux CRT
               colrm : supprime des colonnes d'un texte
POURQUOI    : Nettoyer des sorties de commandes contenant des séquences d'échappement.
CONTEXTE    : Post-traitement de sorties de commandes, nettoyage de texte.
OPTIONS colrm :
  N          Supprime depuis la colonne N jusqu'à la fin
  N M        Supprime les colonnes N à M
EXEMPLES     :
  $ man ls | col -b > ls_propre.txt
  $ echo "abcdef" | colrm 3 4

```